HCMIU Map Collaborative

A Multi-Model Database-Driven Campus Map
with Real-Time Collaboration

Huynh Tran Khanh
Student ID: MITIU25210

Advanced Database System (IT502IU)
International University — VNU-HCM

March 13, 2026

Outline

Problem Context

Challenge: Campus information systems must handle

- **High schema variability** — rooms, notes, comments, trials, moderation records evolve over time
- **High relationship density** — references, follows, replies, dispute participants form a dense graph

Traditional approach: Polyglot persistence (separate document store + graph DB) $\Rightarrow$ operational overhead, synchronisation burden [1, 2]

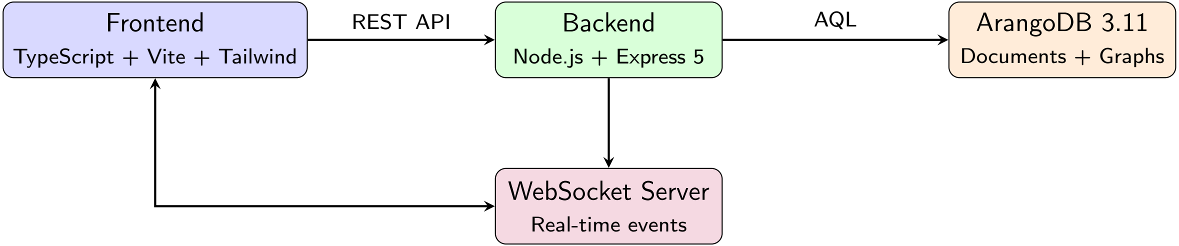
Our Approach

Use **ArangoDB**—a single multi-model engine that natively supports documents, graphs, and key-value workloads—to reduce impedance mismatch while keeping full graph query power.

Project Objectives

1. **Interactive campus navigation** — 7-floor indoor map with BFS shortest-path and Traveling Salesman (Held–Karp) multi-stop optimiser
2. **Collaborative knowledge graph** — entity-based discussions with cross-references, follows, notifications, and an activity feed
3. **Transparent dispute resolution** — a structured "Court of Justice" trial system with judge negotiation and voting
4. **Graph-native research tools** — reference discovery, full-text search, degree-of-separation queries
5. **Real-time experience** — WebSocket event propagation for live updates

System Architecture



- **Deployment:** Docker Compose (3 services: DB, backend, frontend)
- **Frontend:** Hyperscript DOM library + Tailwind CSS + Anime.js
- **Auth:** Client-side Argon2i (libsodium) → server SHA-256

Multi-Model Database Design

Document Collections:

- `users` — credentials, salts, tokens
- `entities` — posts, comments, map locations, trial entities
- `follows` — user $\rightarrow$ entity subscriptions
- `notifications` — per-user alerts
- `trials` — court cases with status, judges, votes

Edge Collection:

- `entity_references` — directed edges connecting entities that reference each other

Why Multi-Model?

- Document storage for flexible schema evolution
- Graph edges for relationship traversal
- Single AQL query language for both
- No cross-system synchronisation needed

Representative AQL Queries

1. Reference Discovery (Graph Traversal):

```
FOR v, e IN 1..1 OUTBOUND @entityId entity_references
  RETURN { entity: v, edge: e }
```

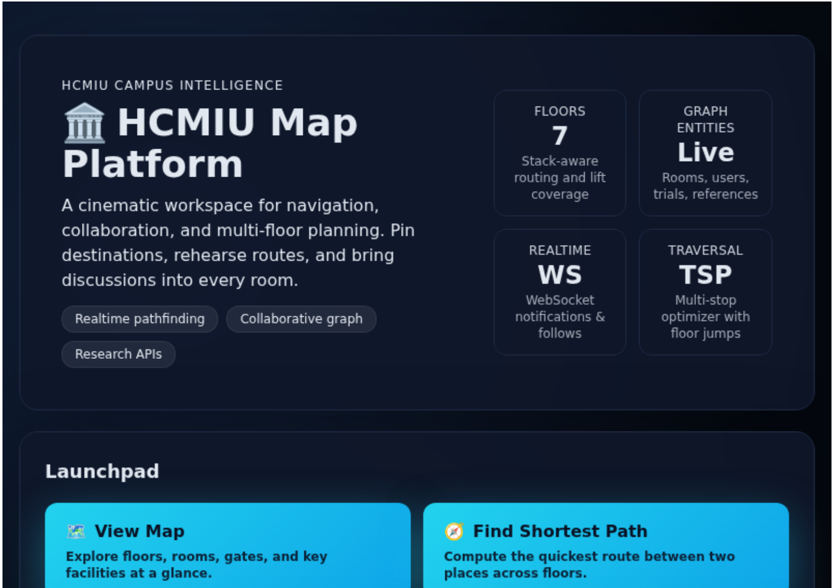
2. Degree of Separation (Shortest Path):

```
FOR v IN OUTBOUND SHORTEST_PATH
  @fromId TO @toId entity_references
  RETURN v
```

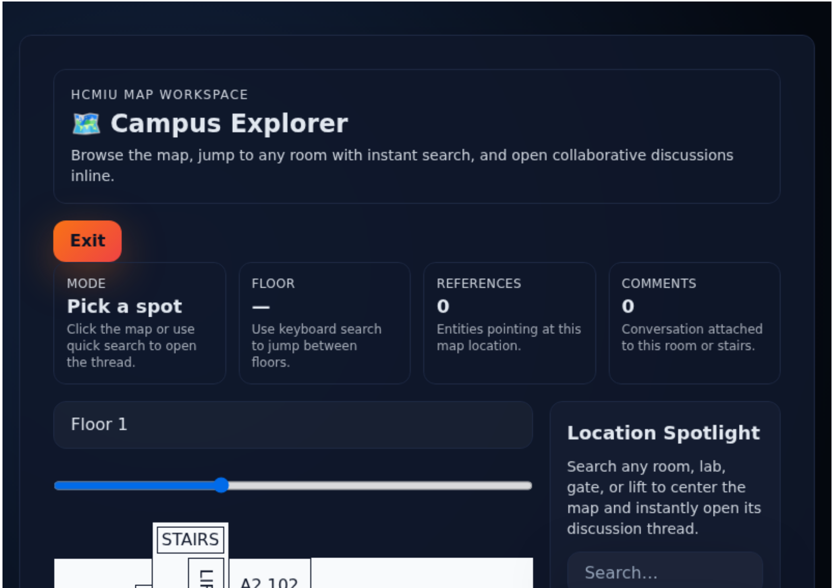
3. Full-Text Search:

```
FOR e IN entities
  FILTER CONTAINS(LOWER(e.title), LOWER(@query))
      OR CONTAINS(LOWER(e.body), LOWER(@query))
  RETURN e
```

Landing Page



Interactive Campus Map



Pathfinding Algorithms

Shortest Path (BFS)

- Breadth-first search on grid graph
- Handles **inter-floor routing** via stairs and lifts
- Visual path overlay on floor maps
- Time complexity: $O(V + E)$

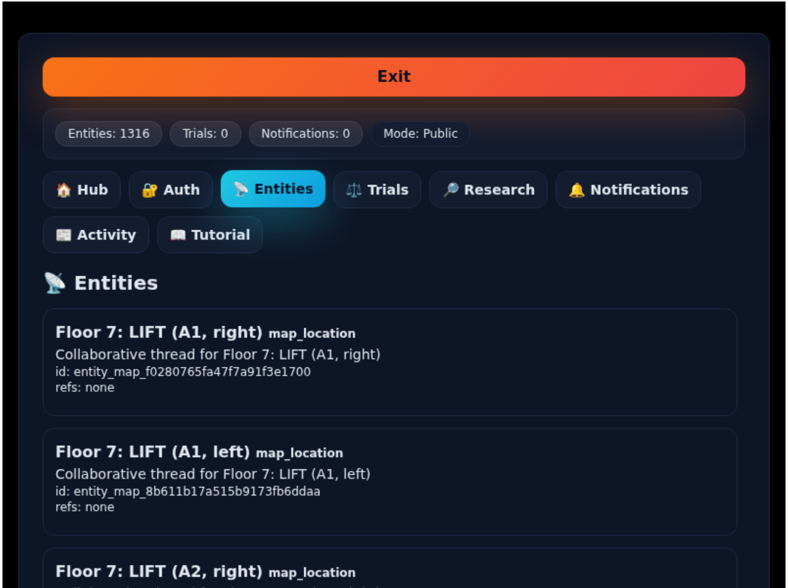
User Flow:
Select start $\rightarrow$ Select end $\rightarrow$
View floor-by-floor directions

Traveling Salesman (Held–Karp)

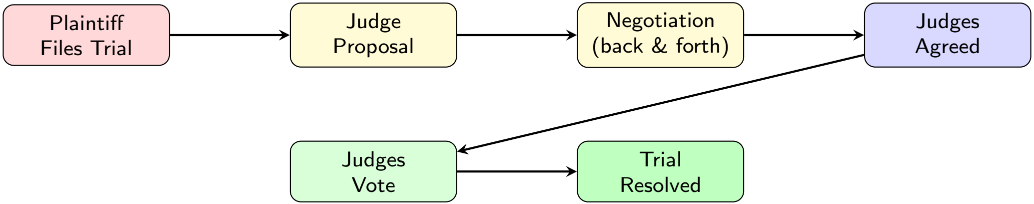
- Bitmask dynamic programming
- Optimal visit order for n destinations
- Multi-floor route planning
- Time complexity: $O(2^n \cdot n^2)$

User Flow:
Add locations $\rightarrow$ Solve $\rightarrow$
View optimised route with total distance

Collaborative Knowledge Graph



Court of Justice — Trial System



- Plaintiff files trial against defendant over a disputed entity
- Both parties negotiate judge selection (proposal $\leftrightarrow$ counter-proposal)
- Agreed judges cast votes: `plaintiff` / `defendant` / `no_winner`
- Comments tagged by role: `[plaintiff]`, `[defendant]`, `[judge]`, `[spectator]`

Research & Knowledge Graph Tools

Reference Discovery

- Query entities by ID
- View all outbound references
- Explore the citation network

Full-Text Search

- Search across all entity titles and bodies
- Keyword-based discovery
- Cross-entity relationship finding

Degree of Separation

- Find shortest reference path between any two entities
- Uses ArangoDB's native `SHORTEST_PATH` function
- Visualises intermediate hops

Mention Discovery

- Find which entities mention a given entity
- Reverse reference traversal
- Map cross-entity relationships

Real-Time Updates via WebSocket

Event Types:

- `entity.created` — new post or comment
- `entity.updated` — edit to existing entity
- `entity.deleted` — entity removed
- `trial.created` — new trial filed
- `trial.updated` — vote or status change
- `notification.created` — user alert

Design Decisions

- RFC 6455 WebSocket protocol
- Token-authenticated connections
- Server-push model (no polling)
- All connected clients auto-refresh

Result: When User A posts a comment, User B sees it instantly without page refresh—verified in multi-user E2E tests.

Technology Stack & Testing

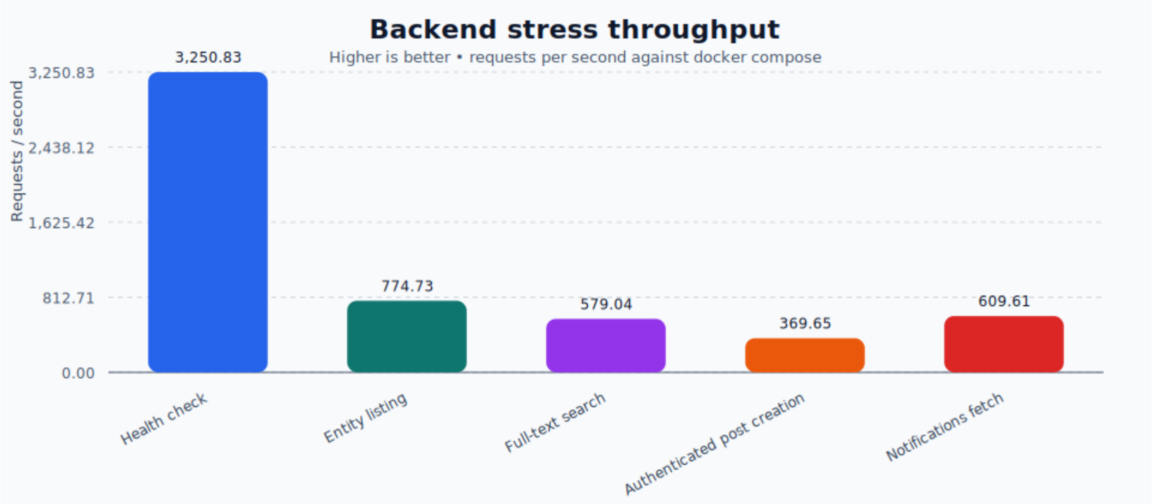
Technology Stack:

Layer	Technology
Frontend	TypeScript, Vite, Tailwind
UI	Hyperscript, Anime.js
Backend	Node.js, Express 5
Database	ArangoDB 3.11
Real-time	WebSocket (ws)
Auth	libsodium (Argon2i)
Deploy	Docker Compose

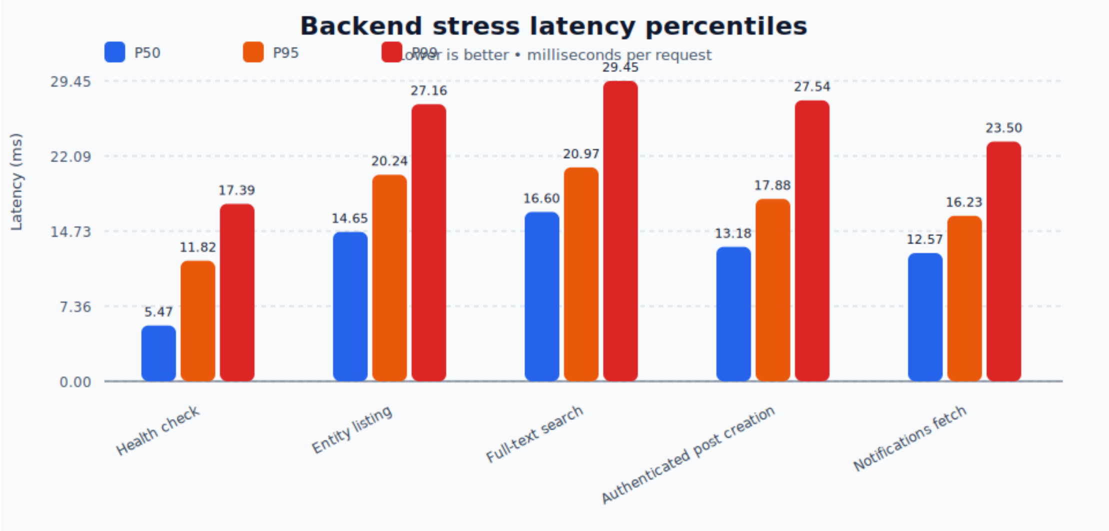
Testing & Validation:

- **API unit tests** — all REST endpoints
- **E2E tests** — full Docker Compose stack
- **Backend stress benchmark visualisations** — throughput and latency charts
- **8 demo videos** — Puppeteer-automated screen recordings covering:
  - Map navigation
  - Multi-user real-time
  - Trial system workflow
  - Research tools (4 demos)
  - General platform overview

Benchmarking: Throughput by Scenario



Benchmarking: Latency Percentiles by Scenario



Conclusions & Future Work

Key Contributions:

1. Demonstrated that a **multi-model DBMS** (ArangoDB) can effectively serve both document storage and graph traversal needs in a single engine
2. Built a **complete campus platform** combining wayfinding, collaboration, governance, and research
3. Implemented **graph-native features** (reference discovery, degree-of-separation, mention traversal) that would be complex in a document-only or relational system
4. Validated correctness through **comprehensive automated testing**

Limitations & Future Work:

- Access control is creator-based; role-based permissions needed
- Event ordering relies on eventual consistency; formal guarantees desirable
- Map data is hardcoded; a map editor would improve extensibility
- Extend benchmarking to sustained mixed-write workloads and larger datasets

References

- M. Stonebraker and U. Çetintemel, "One Size Fits All: An Idea Whose Time Has Come and Gone," *Proc. ICDE*, pp. 2–11, 2005.
- P. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*, Addison-Wesley, 2012.
- ArangoDB Inc., "ArangoDB—the Multi-Model NoSQL Database," `https://www.arangodb.com/`, 2024.
- A. Zimmerman and M. Martin, "Post-occupancy evaluation: benefits and barriers," *Building Research & Information*, vol. 29, no. 2, pp. 168–174, 2001.

Thank You

Questions?

Live Demo: `https://hcmiumap.huynhtrankhanh.com/`

Source Code: `https://github.com/huynhtrankhanh/hcmiu-map-collaborative`